Experiment-6

Student Name: Astik Joshi

Branch: BE-CSE

Semester: 6th

Subject Name: System Design

UID :23BCS10627

Section/Group: KRG-3B

Date of Performance: 27/02/2026

Subject Code: 23CSH-314

Aim :

To design a highly scalable video streaming platform similar to YouTube that supports heavy video uploading, adaptive bitrate streaming, and real-time social interactions with high availability and low latency.

Objectives:

- To study the architectural components of large-scale media streaming platforms.
- To define essential functional requirements such as direct cloud-based uploads and customized content feeds.
- To determine critical non-functional requirements including CDN-based caching, throughput optimization, and latency minimization.
- To examine asynchronous, event-driven processing mechanisms for decoupling synchronous API operations.
- To design RESTful service endpoints for video lifecycle management, discovery, and user engagement.

Procedure

- Evaluated the Pre-Signed URL upload strategy to eliminate API layer bottlenecks during large media transfers.
- Documented both functional and quality-of-service requirements for large-scale content delivery.
- Designed REST APIs for upload orchestration, metadata access, search functionality, and engagement workflows.
- Assessed adaptive streaming protocols (HLS/DASH) and optimized media asset storage by isolating thumbnail resources.
- Investigated real-time stream processing frameworks (Apache Flink with Kafka) for scalable and accurate view-count aggregation.

Functional Requirements

- The platform must enable user registration, authentication, and profile management.
- The system must efficiently handle large raw video uploads.
- The platform must deliver adaptive bitrate streaming based on user network conditions.
- The system must facilitate instant social engagement features including likes, dislikes, and commenting.
- Users must be able to search videos using titles and associated metadata.
- The platform must generate a personalized homepage feed driven by user watch history.

Core Entities of the System

- Users
- Videos
- Interactions (Likes/Dislikes)
- Comments
- ViewHistory

API Design

1. Authentication API

A. **Register**: POST /api/v1/auth/register
B. **Login**: POST /api/v1/auth/login

2. Video & Feed API

A. **Initialize Upload**: POST /api/v1/videos/upload/init (Returns S3 Pre-Signed URL)
B. **Get Video Metadata**: GET /api/v1/videos/{video_id}
C. **Get Personalized Feed**: GET /api/v1/feed (Pagination Supported)
D. **Search Videos**: GET /api/v1/search?q={query}

3. Interaction API

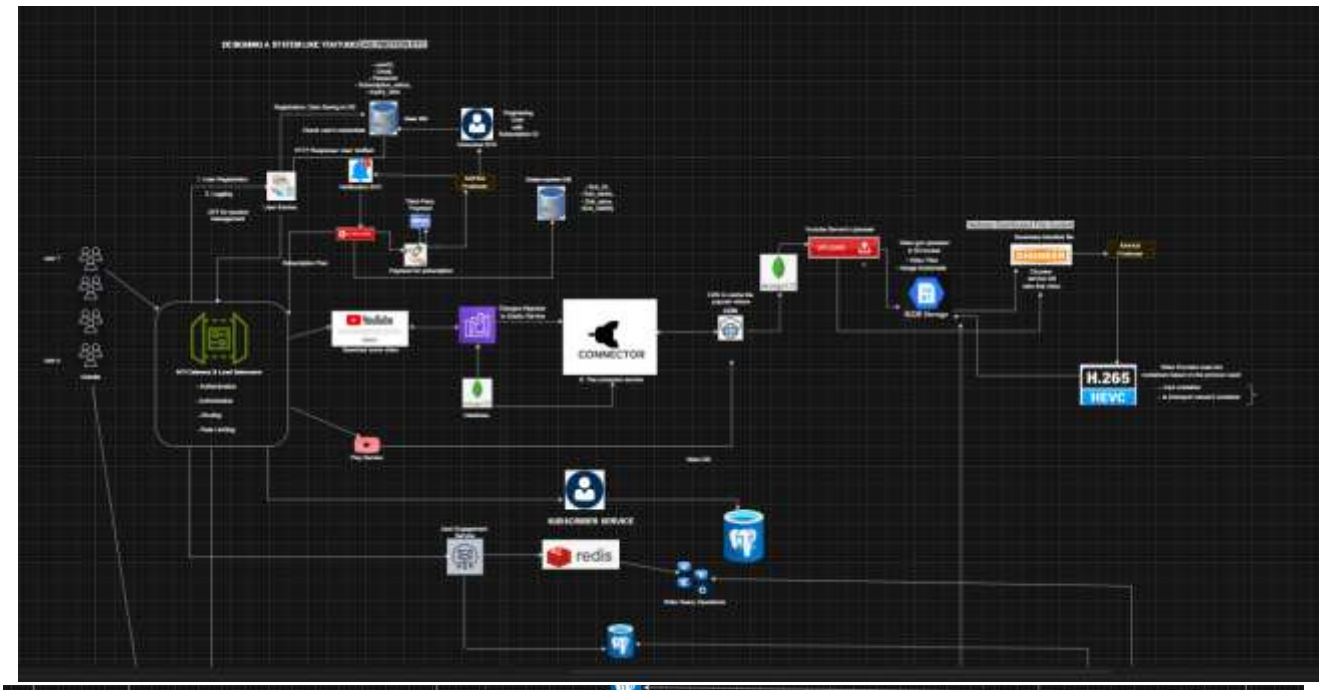
A. **Like Video**: POST /api/v1/videos/{video_id}/like
B. **Add Comment**: POST /api/v1/videos/{video_id}/comments
C. **Get Comments**: GET /api/v1/videos/{video_id}/comments

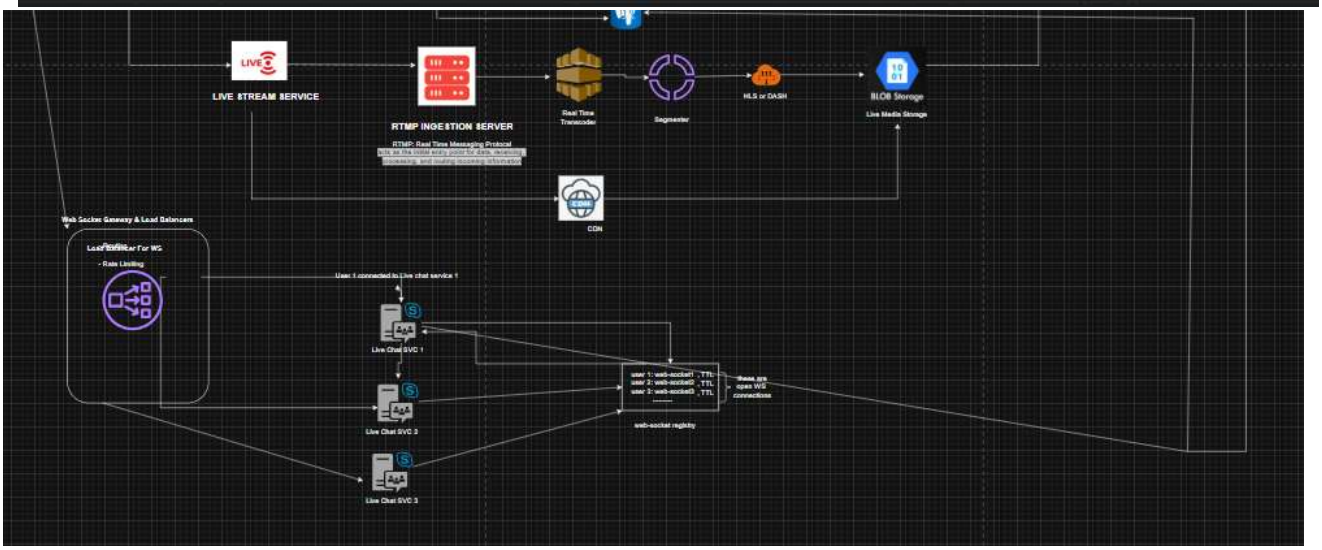
Non-Functional Requirements

- The system must sustain extremely high concurrency levels for uploads and millions of parallel video streams.
- The architecture must ensure high availability with no single point of failure.
- The platform should adopt eventual consistency for engagement metrics (views, likes) to reduce database write pressure.
- Video streaming latency must be minimized using edge caching and globally distributed CDNs.
- The transcoding workflow must scale horizontally to accommodate sudden viral traffic spikes.

High Level Design (HLD)

The platform comprises client interfaces, an API Gateway, and modular microservices such as Authentication, Upload Management, Metadata Service, Feed Service, and Interaction Service. Large video files are uploaded directly to object storage using Pre-Signed URLs, avoiding backend throughput limitations. An asynchronous transcoding pipeline converts raw videos into HLS/DASH segments. Processed content is distributed globally via CDN infrastructure. High-frequency engagement operations are optimized using Redis for like tracking, NoSQL databases for comments, and a Kafka-to-Apache Flink streaming pipeline for scalable and reliable view aggregation.





Outcome

- Developed a scalable, event-driven architecture for large-scale video streaming.
- Clearly defined both functional capabilities and system quality attributes.
- Designed RESTful APIs with asynchronous processing workflows.
- Gained insight into CDN-based delivery, adaptive bitrate streaming, and distributed stream processing in high-traffic environments.